

下面是按你要求整理的 **Markdown** 笔记版本：默认 Ubuntu 虚拟机没有安装 Docker，从零开始安装 Docker，再通过 Docker 配置 6380、6381 两个 Redis 环境。Docker 官方推荐在 Ubuntu 上先配置 Docker 的 apt 仓库，再安装 `docker-ce`、`docker-ce-cli`、`containerd.io` 等组件。(Docker Documentation)

8.1 本节目标

本节用于 Redisson 主从一致性问题解决课程环境准备，通过 Docker 在 Ubuntu 中额外启动两个 Redis 实例。

```
# 最终目标：
# 原本已有 Redis: 6379
# Docker 新增 Redis: 6380
# Docker 新增 Redis: 6381

# 最终 Java 连接地址类似：
# redis://192.168.10.101:6379
# redis://192.168.10.101:6380
# redis://192.168.10.101:6381

# 注意：
# 这里不是使用 Redis 的 db0、db1、db2
# 而是要启动多个独立 Redis 实例
```

8.2 默认当前环境没有安装 Docker

先检查系统中是否已经安装 Docker，如果提示找不到命令，就说明需要从零安装。

```
docker --version
# 查看 Docker 版本
# 如果输出 Docker version xxx, 说明已经安装
# 如果提示 command not found, 说明 Docker 没有安装

docker -v
# docker --version 的简写形式
# 作用也是查看 Docker 版本
```

8.3 卸载可能冲突的旧版本

Ubuntu 系统里可能自带旧版 Docker 包，为了避免和 Docker 官方版本冲突，先统一卸载旧版本。

```
sudo apt remove -y docker.io docker-doc docker-compose docker-  
compose-v2 podman-docker containerd runc  
# sudo: 使用管理员权限执行命令  
# apt remove: 卸载软件包  
# -y: 自动确认，不需要手动输入 yes  
# docker.io、docker-doc、docker-compose 等：可能存在的旧版本或冲突包  
# 如果提示某些包没有安装，是正常现象，不影响后续操作
```

8.4 更新 apt 并安装基础依赖

安装 Docker 前，需要先更新软件源，并安装 HTTPS 下载、证书相关依赖。

```
sudo apt update  
# 更新 Ubuntu 软件包索引  
# 让系统知道当前软件源中有哪些可安装的软件版本  
  
sudo apt install -y ca-certificates curl  
# 安装 Docker 官方源所需的基础工具  
# ca-certificates: 用于 HTTPS 证书校验  
# curl: 用于下载 Docker 官方 GPG key  
# -y: 自动确认安装
```

8.5 添加 Docker 官方 GPG key

GPG key 用来校验 Docker 软件包来源，保证 apt 下载的软件包是可信的。

```
sudo install -m 0755 -d /etc/apt/keyrings  
# 创建 /etc/apt/keyrings 目录  
# install -d: 创建目录  
# -m 0755: 设置目录权限  
# 该目录用于存放 Docker 的 GPG key
```

```
sudo curl -fSSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
# 从 Docker 官方地址下载 GPG key
# curl: 下载文件
# -f: 请求失败时直接报错
# -s: 静默模式, 减少输出
# -S: 出错时显示错误信息
# -L: 自动跟随重定向
# -o: 指定保存路径

sudo chmod a+r /etc/apt/keyrings/docker.asc
# 给 docker.asc 增加所有用户可读权限
# apt 后续读取 Docker 源时需要访问这个 key 文件
```

8.6 添加 Docker 官方 apt 软件源

添加 Docker 官方软件源后, Ubuntu 才能通过 `apt install` 安装 Docker 官方版本。

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo
"${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
# tee: 把内容写入文件
# /etc/apt/sources.list.d/docker.sources: Docker 的 apt 源配置文件
# URIs: Docker 官方 Ubuntu 软件源地址
# Suites: 自动读取当前 Ubuntu 版本代号, 比如 focal、jammy、noble
# Components: stable: 使用稳定版 Docker 源
# Architectures: 自动读取当前系统架构, 比如 amd64
# Signed-By: 指定刚才下载的 Docker GPG key

sudo apt update
# 重新更新软件源
# 让 Ubuntu 识别刚刚新增的 Docker 官方 apt 源
```

8.7 安装 Docker

安装 Docker Engine、命令行工具、容器运行时、构建插件和 Compose 插件。

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin
# docker-ce: Docker 社区版服务端
# docker-ce-cli: Docker 命令行客户端
# containerd.io: 容器运行时, Docker 依赖它管理容器
# docker-buildx-plugin: Docker 构建插件
# docker-compose-plugin: Docker Compose 插件
# -y: 自动确认安装

docker --version
# 安装完成后查看 Docker 版本
# 如果能看到 Docker version xxx, 说明 Docker 安装成功
```

8.8 启动 Docker 服务并设置开机自启

Docker 安装完成后, 需要启动 Docker 服务, 否则无法运行容器。

```
sudo systemctl start docker
# 启动 Docker 服务

sudo systemctl enable docker
# 设置 Docker 开机自启
# 避免虚拟机重启后 Docker 服务没有自动启动

sudo systemctl status docker
# 查看 Docker 服务状态
# 如果看到 active (running), 说明 Docker 正在运行

docker ps
# 查看当前正在运行的容器
# 如果 Docker 正常运行, 即使没有容器, 也不会报错
```

8.9 配置 Docker 镜像加速

如果拉取 `redis:7` 镜像很慢，或者出现网络超时、TLS handshake 等问题，可以配置镜像加速。

```
sudo mkdir -p /etc/docker
# 创建 Docker 配置目录
# -p 表示目录已存在时不报错

sudo tee /etc/docker/daemon.json <<EOF
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io"
  ]
}
EOF
# 创建或覆盖 Docker 配置文件 daemon.json
# registry-mirrors: 配置 Docker 镜像加速地址
# 注意: JSON 文件中不能写 # 注释，所以解释写在命令外面

sudo systemctl daemon-reload
# 重新加载 systemd 配置

sudo systemctl restart docker
# 重启 Docker 服务
# 让 daemon.json 配置生效

docker info
# 查看 Docker 详细信息
# 可以检查 Registry Mirrors 是否配置成功
```

8.10 拉取 Redis 7 镜像

Redis 容器需要基于 Redis 镜像启动，`redis:7` 表示 Redis 官方镜像的 7.x 版本。

```
docker pull redis:7
```

```
# docker pull: 拉取镜像
```

```
# redis: 镜像名称, 表示 Redis 官方镜像
```

```
# 7: 镜像版本标签, 表示 Redis 7.x 版本
```

```
docker images
```

```
# 查看本地已有镜像
```

```
# 如果看到 redis 7, 说明 Redis 镜像已经下载成功
```

8.11 启动 6380 Redis 容器

启动第一个 Docker Redis 容器, 对外暴露端口为 6380。

```
docker run -d --name redis-6380 -p 6380:6379 redis:7
```

```
# docker run: 创建并启动容器
```

```
# -d: 后台运行容器
```

```
# --name redis-6380: 指定容器名称为 redis-6380
```

```
# -p 6380:6379: 端口映射
```

```
# redis:7: 使用 Redis 7 镜像启动容器
```

```
# 重点理解:
```

```
# 6380:6379
```

```
# 左边 6380 是 Ubuntu 虚拟机暴露给外部访问的端口
```

```
# 右边 6379 是 Redis 容器内部端口
```

```
# Redis 容器内部默认端口仍然是 6379
```

```
# 但是 Java 程序访问时要连接宿主机的 6380
```

8.12 启动 6381 Redis 容器

启动第二个 Docker Redis 容器, 对外暴露端口为 6381。

```
docker run -d --name redis-6381 -p 6381:6379 redis:7
# docker run: 创建并启动容器
# -d: 后台运行容器
# --name redis-6381: 指定容器名称为 redis-6381
# -p 6381:6379: 把宿主机 6381 映射到容器内部 6379
# redis:7: 使用 Redis 7 镜像

# 重点理解:
# redis-6380 容器: 宿主机 6380 -> 容器内部 6379
# redis-6381 容器: 宿主机 6381 -> 容器内部 6379
```

8.13 查看 Redis 容器是否运行成功

启动完成后，通过 `docker ps` 查看当前正在运行的容器。

```
docker ps
# 查看正在运行的容器

# 正常情况下应该能看到类似内容:
# redis-6380    0.0.0.0:6380->6379/tcp
# redis-6381    0.0.0.0:6381->6379/tcp

# 0.0.0.0:6380->6379/tcp 表示:
# 访问 Ubuntu 虚拟机的 6380 端口，会进入 redis-6380 容器内部的 6379 端口

# 0.0.0.0:6381->6379/tcp 表示:
# 访问 Ubuntu 虚拟机的 6381 端口，会进入 redis-6381 容器内部的 6379 端口
```

8.14 测试 6380 Redis 是否可用

可以进入 `redis-6380` 容器内部，使用 `redis-cli` 测试 Redis 是否正常。

```
docker exec -it redis-6380 redis-cli
# docker exec: 进入正在运行的容器执行命令
# -it: 交互式终端
# redis-6380: 容器名称
# redis-cli: Redis 命令行客户端
```

ping

```
# 在 redis-cli 中输入 ping
# 如果返回 PONG, 说明 Redis 正常运行
```

exit

```
# 退出 redis-cli
```

8.15 测试 6381 Redis 是否可用

同样进入 `redis-6381` 容器内部, 测试 Redis 是否正常。

```
docker exec -it redis-6381 redis-cli
# 进入 redis-6381 容器内部的 Redis 客户端
```

ping

```
# 测试 Redis 是否响应
# 正常返回 PONG
```

exit

```
# 退出 Redis 客户端
```

8.16 使用宿主机端口测试 Redis

如果 Ubuntu 中安装了 `redis-cli`, 也可以直接通过宿主机端口测试。

```
redis-cli -h 127.0.0.1 -p 6380
# -h 127.0.0.1: 连接本机
# -p 6380: 连接宿主机暴露出来的 6380 端口
# 实际会进入 redis-6380 容器内部的 6379
```

ping


```
# 正常返回 PONG

exit
# 退出 redis-cli

redis-cli -h 127.0.0.1 -p 6381
# 连接宿主机 6381 端口
# 实际会进入 redis-6381 容器内部的 6379

ping
# 正常返回 PONG

exit
# 退出 redis-cli
```

8.17 如果提示容器名称已存在

如果之前启动过容器，再次创建同名容器时，可能会提示容器名冲突。

```
docker ps -a
# 查看所有容器
# 包括正在运行的容器和已经停止的容器

docker rm -f redis-6380
# 删除 redis-6380 容器
# -f 表示强制删除，如果容器正在运行，会先停止再删除

docker rm -f redis-6381
# 删除 redis-6381 容器

docker run -d --name redis-6380 -p 6380:6379 redis:7
# 重新创建并启动 redis-6380

docker run -d --name redis-6381 -p 6381:6379 redis:7
# 重新创建并启动 redis-6381
```

8.18 如果提示 Unable to find image redis:7 locally

这句话本身不是最终错误，它只是表示本地没有 Redis 镜像，Docker 准备去远程仓库下载。

```
docker run -d --name redis-6380 -p 6380:6379 redis:7
# 如果出现 Unable to find image 'redis:7' locally
# 说明本地没有 redis:7 镜像
# Docker 会自动尝试从远程仓库下载

docker pull redis:7
# 如果自动下载失败，可以手动拉取 Redis 镜像

docker images
# 查看 redis:7 是否已经下载到本地

docker run -d --name redis-6380 -p 6380:6379 redis:7
# 镜像下载成功后，再重新启动容器
```

8.19 如果 Docker 服务没有启动

如果执行 Docker 命令时报 Docker daemon 相关错误，通常是 Docker 服务没有启动。

```
systemctl status docker
# 查看 Docker 服务状态
# 如果不是 active (running)，说明 Docker 没有正常运行

sudo systemctl start docker
# 启动 Docker 服务

sudo systemctl restart docker
# 重启 Docker 服务
# 修改 daemon.json 后通常需要执行这个命令

docker ps
# 再次测试 Docker 是否能正常使用
```

8.20 最终 Redis 环境结构

完成后，Ubuntu 虚拟机中应该有三个 Redis 实例，可以用于 Redisson MultiLock 实验。

```
# 原来的 Redis:
# 192.168.10.101:6379

# Docker 新增的 Redis:
# 192.168.10.101:6380
# 192.168.10.101:6381

docker ps
# 查看当前 Docker Redis 容器
# 应该能看到 redis-6380 和 redis-6381

# 端口关系:
# redis-6380: 宿主机 6380 -> 容器内部 6379
# redis-6381: 宿主机 6381 -> 容器内部 6379

# Java 中连接时使用宿主机端口:
# redis://192.168.10.101:6379
# redis://192.168.10.101:6380
# redis://192.168.10.101:6381
```

8.21 Redisson MultiLock 对应关系

Docker 环境准备完成后，Java 代码中需要配置三个 `RedissonClient`，分别连接三个 Redis 实例。

```
// 6379 Redis 上的 order 锁
RLock lock1 = redissonClient6379.getLock("order");

// 6380 Redis 上的 order 锁
RLock lock2 = redissonClient6380.getLock("order");

// 6381 Redis 上的 order 锁
RLock lock3 = redissonClient6381.getLock("order");

// 把三把锁组合成一把 MultiLock
```

```
RLock multiLock = redissonClient6379.getMultiLock(lock1, lock2,
lock3);

// MultiLock 加锁成功的含义:
// 6379 上的 order 锁成功
// 6380 上的 order 锁成功
// 6381 上的 order 锁成功
// 三个 Redis 实例都加锁成功, MultiLock 才算成功
```

8.22 本节总结

本节核心是使用 Docker 在 Ubuntu 中额外启动两个 Redis 实例，给 Redisson MultiLock 实验准备环境。

```
# 本节最核心的两个命令:

docker run -d --name redis-6380 -p 6380:6379 redis:7
# 启动 redis-6380 容器
# 宿主机端口 6380 映射到容器内部 Redis 默认端口 6379

docker run -d --name redis-6381 -p 6381:6379 redis:7
# 启动 redis-6381 容器
# 宿主机端口 6381 映射到容器内部 Redis 默认端口 6379

# 最终形成三个 Redis 实例:
# 6379: 原 Redis
# 6380: Docker Redis
# 6381: Docker Redis

# Redisson MultiLock 本质:
# 不是一个 Redis 多个 db
# 而是多个 Redis 实例分别加锁
# 再把多把 RLock 组合成一把 MultiLock
```